Machine Learning Lab
Week 16 February to 22 February

1. Implement the closed-form solution for Ridge Regression: $\hat{\beta} = (X^T X + \lambda I)^{-1} X^T y$. Using a dataset with high multicollinearity (like the *Boston Housing* or *Diabetes* dataset), plot how the magnitude of the coefficients (β weights) changes as λ increases from 0 to 10^4.

2. Generate a synthetic dataset with a small number of samples and significant noise. Train multiple Ridge models with varying λ values. Plot the Training MSE and Test MSE on the same graph. Identify the "Sweet Spot" and explain why Ordinary Least Square ($\lambda = 0$) fails in this scenario.

3. Perform k-fold cross-validation from scratch to find the optimal λ for a Ridge Regression model. Compare your "hand-coded" optimal λ with the one selected by RidgeCV from Scikit-Learn.

4. Create a non-linearly separable 2D dataset (e.g., concentric circles or "moons").

a) Attempt to classify it using a Linear SVM.

b) Implement a Gaussian RBF Kernel to project the data into a higher dimension.

c) Visualize the decision boundary in the original 2D space.

5. Using the *UCI Wine* dataset, compare the classification accuracy of an SVM using three different kernels: Linear, Polynomial (degree 3), and RBF**.** Which one performs best? Hypothesize why based on the feature distribution.

6. For an RBF Kernel SVM, create a heatmap showing the validation accuracy for different combinations of C (Regularization) and γ (Kernel width). Explain what happens to the decision boundary when γ is set too high.

7. In many libraries, you can define your own kernel. Implement a custom Laplacian Kernel $K(x, y) = e^{-\alpha \|x - y\|_1}$ and use it within an SVM framework. Compare its performance against the standard RBF kernel on a dataset of your choice.

8. A single-layer Perceptron cannot solve the Exclusive-OR (XOR) problem.

- Prove this by attempting to train a Perceptron on XOR data.
- Build a Multi-Layer Perceptron (MLP) with one hidden layer (2-4 neurons) that successfully classifies XOR. Plot the decision boundary.

9. Implement a simple 3-layer Neural Network (Input, Hidden, Output) using only NumPy. You must manually code:

a) The forward pass.

b) The loss function (MSE or Cross-Entropy).

c) The backpropagation algorithm using the chain rule to update weights.

10. Train an MLP on the *MNIST* digits dataset. Run three separate trials using Sigmoid, Tanh, and ReLU activation functions in the hidden layers. Plot the loss curves for each over 50 epochs. Which one converges the fastest, and why?

11. Experiment with weight initialization. Compare the training progress of a deep network (5+ layers) when weights are initialized to all zeros and large random numbers. Observe and document the "vanishing gradient" problem.

12. Take a standard MLP and tune the following:

- Number of hidden layers vs. Neurons per layer.
- Learning rate (try $0.1, 0.01, 0.001$).
- Batch size (Stochastic vs. Mini-batch vs. Batch Gradient Descent).

Summarize which parameter had the most significant impact on final model accuracy.